Usability Evaluation

Heuristic Evaluation

We used established usability heuristics to evaluate the software's interface. The following usability principles were considered:

H1: Visibility of System Status:
Our system ensures that the user always knows what is happening, by notifying them of their current situation/status at every stage.

Example:

- menu.py line 30 ‘Utils.display_str("Successfully logged in as a Learner!")’ allows the user to know that they have successfully logged in and as which user type.

- menu.py line 117 ‘print("Invalid input. Please enter a valid integer.")’ allows the user to know of an error.

H2: Match Between System and the Real World:
Our system maintains a strong match between the system and the real world. It uses language and concepts that are familiar and natural to the user. The words and phrases used in the system, including menu options and error messages, are designed to mimic real-world communication. For example, the system prompts users to "Select a menu option" using clear and commonly understood language. This alignment with real-world concepts aids in making the system intuitive and easy to comprehend.

H3: User Control and Freedom:
Our system is in the process of implementing the user control and freedom heuristic. While not yet fully implemented, we recognize the importance of allowing users to leave unwanted states or incorrect selections quickly. In the current command-line interface, users can return to the main menu and log out to regain control or recover from undesired states. As the system evolves, additional features will be added to enhance this aspect of user control and freedom, ensuring users have the flexibility to navigate the system efficiently.

H4: Consistency and Standards:
Our system places a strong emphasis on consistency and adheres to established standards throughout the user interface. We have designed the system to ensure that users encounter consistent behaviours and user interface elements, eliminating any confusion or ambiguity. For example, the main menu offers consistent options, such as "Login," "Register," and "Turn Off," ensuring users can predict the outcome of each action. Error messages are consistently formatted and presented, enabling users to quickly recognize and understand the nature of the error. The system also maintains a consistent language and interaction pattern, which simplifies user understanding and interaction. Whether registering as a "Learner" or a "Parent," users encounter the same standard prompts and expectations, ensuring a consistent and predictable registration process.

H5: Error Prevention:

Our system notifies the user of expected input and its format. For example, our menu notifies users that their input should strictly be among the numbers displayed.

Our challenges also ensure that our input prompts are expressive enough that users know what to input. An example can be seen below:
“question": "Declare a function called add_numbers which takes in two arguments, num1 and num2. Write code until the colon, including it. ",

H6: Recognition Rather than Recall:
Our system ensures that users do not need to memorise or recall information. Instead, it presents options and information directly, reducing the cognitive load on users. For instance, in the login menu, the available options are displayed, and users only need to select from them without needing to remember specific commands or options.

H7: Flexibility and Efficiency of Use:

Our system provides flexibility in terms of user interactions. While our current implementation may not include shortcuts or quick navigation, the software has been designed to be user-friendly and accessible to all users, regardless of their experience level. Users can efficiently navigate through the menu structure and access various features in a straightforward manner. This approach prioritises ease of use and ensures that all users can interact with the software effectively, even without advanced features like shortcuts.

H8: Aesthetic and Minimalist Design:
Our current system design prioritises readability, user guidance, and efficient navigation, ensuring that users can interact with the application seamlessly while keeping the interface clean and uncluttered. By doing so, users are provided with an elegant and distraction-free environment to engage with its educational content and functionality.

When migrating to GUI from the terminal, our design will prioritise simplicity and functionality, ensuring a clutter-free, visually appealing user experience. With a clean and intuitive interface, limited colour palette, thoughtful typography, and consistent design elements, the application will offer users, especially young learners, a user-friendly platform that reduces distractions and enhances usability.

H9: Help Users Recognize, Diagnose, and Recover from Errors:
Our system is designed to help users promptly recognize, diagnose, and recover from errors. Whenever an error occurs, our system provides clear and informative error messages that guide users to understand the issue and take corrective actions.

For example, when a user inputs an invalid integer in the main menu, an error message is displayed, stating, "Invalid input. Please enter a valid integer” (menu.py). This message not only informs users of the error but also offers a clear suggestion on how to rectify it. By providing such actionable error feedback, our system enables users to quickly diagnose and recover from errors, reducing frustration and facilitating a smoother user experience.

H10: Help and documentation:

Our system does not explicitly provide help and documentation. Our system has been designed with a focus on being intuitive and expressive, aiming to minimise the need for extensive help and documentation. While we have strived to make the user interface user-friendly and self-explanatory, we recognize that some users may still benefit from assistance or guidance in certain situations.

Findings
In our heuristic evaluation, we have uncovered critical insights into the software's interface:

Strengths:

- Visibility of System Status (H1): The software effectively conveys user status, such as successful login, through clear messages, enhancing the user experience.

- Match Between System and the Real World (H2): The software utilises familiar language and concepts, making it intuitive and user-friendly.

Areas for improvement:

- User Control and Freedom (H3): While users can return to the main menu and log out, further enhancements to user control are needed.

- Flexibility and Efficiency of Use (H7): Although user-friendly, the software currently lacks shortcuts or quick navigation features, which could significantly improve efficiency.

- Help Users Recognize, Diagnose, and Recover from Errors (H9): The system excels in providing clear and actionable error messages, enhancing user experience by facilitating error recognition and recovery.

- Help and documentation (H10): While the software aims for an intuitive user interface, consideration should be given to users who may require assistance in certain situations.

User Testing

User testing involved 5 participants who interacted with the software. Each participant was provided with scenarios or tasks to complete, and they were encouraged to "think aloud" while using the software. The tasks covered key features and typical user interactions and are as follows:

Scenarios

User Registration:

Scenario: You are a new user interested in using CodeVenture. Your goal is to create an account. Please register as a "Learner" with the username "newLearner" and a password of your choice.
Task: Register a new learner account.

Selecting a Learning Module:

Scenario: You are logged in as a learner. Your objective is to choose a learning module on Python variables.
Task: Navigate through the menu and select the Python Variables module.

Learning Interaction:

Scenario: You are inside the Python Variables module. You want to learn about variables. Please go through the module's content.
Task: Interact with the learning module content and attempt the quiz at the end.

Progress Tracking:

Scenario: As a learner, you want to check your progress. You've completed the Python Variables module, and now you want to view your progress.
Task: Access the progress tracker and check your progress.

Educator Features:

Scenario: You are an educator using CodeVenture. Your goal is to modify an existing learning module.
Task: Access the educator menu and initiate the process of modifying a learning module.

Findings

Areas of Strength:

- Ease of Navigation: Participants appreciated the clear and intuitive menu structure, making it easy to find and access different features. They felt that the software's layout was user-friendly and conducive to a smooth user experience.

- Educational Content: Users found the learning modules engaging and helpful in understanding Python programming concepts. They praised the interactive tutorials and the step-by-step approach to learning.

- User Authentication: The login system was praised for its simplicity and security features, ensuring a seamless login process.

- Progress Tracking: Participants liked the progress tracking feature, allowing them to monitor their learning journey effectively. They found it motivating to see their accomplishments and track their advancement through modules.

Key Issues and Improvements:

- Limited Educator Features: Some participants expressed an interest in additional features for educators, suggesting the ability to customise learning modules and track learner progress more comprehensively.

- Parental Controls: Participants taking the role of parents suggested the inclusion of more extensive parental control options to monitor and manage their child's account effectively.

- User Documentation: Some participants felt that more extensive user documentation or tooltips would be beneficial for first-time users. They suggested that additional guidance and explanations could help new users navigate the platform more easily and take full advantage of its features.

Recommendations

Based on the findings from the heuristic evaluation and user testing, we recommend the following improvements:

Enhance User Control (H3): Provide users with more control features, like a "Back" option for navigation and task reversal capabilities.

Efficiency Features (H7): Introduce shortcut keys or quick navigation options for improved efficiency.

Guided Onboarding (H10): Consider offering guided onboarding for new users to familiarise them with the platform.

Educator and Parental Enhancements: Extend educator features and provide parents with more account control options.

User Documentation: Develop user documentation or tooltips for improved user guidance.

Strengthen Error Prevention and Recovery (H5, H9): Provide clearer input prompts and enhance error prevention mechanisms.

Conclusion

This assessment of usability has unlocked valuable perspectives on both the strong points and areas needing improvement in CodeVenture. By employing usability heuristics and engaging users in the testing process, we've pinpointed possibilities for improving the user experience. The execution of the suggested enhancements will result in a software solution that is more user-friendly and accessible.

Quality Assurance Plan

QA Strategy

Our Quality Assurance (QA) plan for CodeVenture is designed to ensure that the software meets its functional and non-functional requirements while maintaining a high level of reliability. This plan outlines the scope, estimates the required resources, and provides a comprehensive testing strategy, covering both static and dynamic analyses.

Scope

The QA scope for CodeVenture encompasses a range of testing activities, including unit testing, functional testing, and performance testing. We aim to evaluate the software's functionality, usability and security.

Resources Estimate

To execute our QA strategy effectively, we estimate the following resources:

QA Team:

- QA Engineer: Vincent Tanuwidjaja
- QA Analyst: Braden Shiew
- QA Tester: Choy Sheng Jie
- QA Coordinator: Go Wei Zheng

Test Environment: PyCharm, personal laptops

Test Data:

Testing Tools: PyTest

Testing Plan

Our testing plan involves two main categories: unit testing and functional testing.

Unit Testing

Unit testing focuses on validating individual components and functions within the software. (We will write and run unit tests for all components using PyTest. - wont be doing this as it is quite complicated) The current phase of this project is only the Prototype, hence much of the functionalities are not final and are being worked on toward the final version.

Test Cases

- Positive Testing:

    - Test Case 1: Successful User Registration
      Test Inputs: Valid username and password
      Expected Outcome: User registration is successful
      Actual Outcome: User registration is successful

- Test Case 2: Successful User Login
  Test Inputs: Valid username and password
  Expected Outcome: Show corresponding user type menu
  Actual Outcome: Show corresponding user type menu

- Test Case 3: Correct option selected in learner menu
  Test Inputs: Click one of the displayed option
  Expected Outcome: Runs the corresponding function of the selected option
  Actual Outcome: Runs the corresponding function of the selected option

- Test Case 4: Learner selects a learning module
  Test Inputs: Click one of the displayed learning module
  Expected Outcome: Display a learning module's content
  Actual Outcome: Display a learning module's content

- Test Case 5: Learner chooses to return back to the Learner main menu after deciding not to select a learning module.
  Test Inputs: Picked return to menu option
  Expected Outcome: Return back to Learner menu
  Actual Outcome: Return back to Learner menu

- Test Case 6: Learner attempts quiz
  Test Inputs: Click one of the displayed answer
  Expected Outcome: "Correct" or "Incorrect" based on whether answer given is correct
  Actual Outcome: "Correct" or "Incorrect" based on whether answer given is correct

- Test Case 7: Parent chooses to block his/her child's access to the software
  Test Inputs: Click "Block Access" button
  Expected Outcome: Parent's child will be denied access when attempting to login
  Actual Outcome: Parent's child will be denied access when attempting to login

- Test Case 8: Parent chooses to unblock his/her child's access to the software
  Test Inputs: Click "Unblock Access" button
  Expected Outcome: Parent's child is able to login to the software again
  Actual Outcome: Parent's child is able to login to the software again

● Negative Testing:
  - Test Case 1: User Registration with Invalid Data
    Test Inputs: Invalid username or password, or repeated username
    Expected Outcome: "Invalid username", "Invalid password" or "Username taken" based on the error
    Actual Outcome: "Invalid input! Please select another option: "

- Test Case 2: User Login with Invalid Data
  Test Inputs: Invalid username or password, or both
  Expected Outcome: “User is not registered” or “Wrong password!”
  Actual Outcome: “User is not registered” or “Wrong password!”

- Test Case 3: User tries to start lesson before completing the previous one
  Expected Outcome: “Unable to start test, please complete the previous lesson”
  Actual Outcome: “Selects learning module regardless of whether previous learning module is completed”

- Test Case 4: Learner selects a non-integer value in the learning module menu
  Test Inputs: Non-integer value (e.g., a letter)
  Expected Outcome: “Please enter a valid level”
  Actual Outcome: “Please enter a valid level”

Functional Testing

Functional testing aims to evaluate all functional requirements of CodeVenture. This includes end-to-end testing of user scenarios to simulate real user interactions.

End-to-End Tests:

Test Scenario 1: User Authentication (Login System)

- User Action: The user runs the application and selects the login option.
  Expected Response: The system prompts the user for their username.
  Actual Response: The system prompts the user for their username.

- User Action: The user inputs a non-existing username.
  Expected Response: The system displays an error message and prompts the user again.
  Actual Response: The system displays an error message and prompts the user again.

- User Action: The user inputs a valid username.
  Expected Response: The system prompts the user for their password.
  Actual Response: The system prompts the user for their password.

- User Action: The user inputs an incorrect password.
  Expected Response: The system displays an error message and prompts the user to input the password again.
  Actual Response: The system displays an error message and prompts the user to input the password again.

- User Action: The user inputs a correct password.

Expected Response: The system verifies the user's identity and displays the main menu.
Actual Response: The system verifies the user's identity and displays the main menu.

Test Scenario 2: Navigating the main menu (Menu)

- User Action: A young learner logs in to their account.
  Expected Response: The system displays a numbered list of menu options.
  Actual Response: The system displays a numbered list of menu options.

- User Action: The user selects a specific option from the menu.
  Expected Response: The system proceeds to the corresponding functionality associated with the chosen option.
  Actual Response: The system proceeds to the corresponding functionality associated with the chosen option.

Test Scenario 3: Completing a Learning Module (Learning Modules, Quiz, Challenge)

- User Action: The user logs in to their account and selects a specific learning module.
  Expected Response: The system displays the interactive module's content regarding Python programming concepts.
  Actual Response: The system displays the interactive module's content regarding Python programming concepts.

- User Action: The user interacts with the learning content and answers quiz questions and challenges.
  Expected Response: The system provides immediate feedback on the correctness of the user's answers.
  Actual Response: The system provides immediate feedback on the correctness of the user's answers.

- User Action: The user completes the module and returns to the main menu.
  Expected Response: The system updates the Progress Tracker with the progress of the completed module.
  Actual Response: The system updates the Progress Tracker with the progress of the completed module.

Test Scenario 4: Viewing Progress and Recommendations (Progress Tracker)

- User Action: The user navigates to the Progress Tracker from the menu.
  Expected Response: The system displays all available modules, challenges, and quizzes with their corresponding progress status.
  Actual Response: The system displays all available modules, challenges, and quizzes with their corresponding progress status.

- User Action: The user selects a specific module, challenge, or quiz to view detailed progress.
  Expected Response: The system provides a detailed overview of the user's progress within the selected item and displays a "Recommended Next Step" based on the user's progress.

Actual Response: The system provides a detailed overview of the user's progress within the selected item and displays a "Recommended Next Step" based on the user's progress.

Use of Techniques:
Equivalence Classes: Equivalence classes are employed to ensure that various categories of inputs are tested to cover a range of scenarios. For example, we use equivalence classes for both positive and negative testing in the unit testing section. In the positive testing cases, valid username and password inputs fall into one equivalence class, while valid integer values for menu options and quiz answers belong to another. In the negative testing cases, inputs that result in errors, such as invalid usernames or repeated usernames, form distinct equivalence classes.

Boundary Value Analysis: Boundary value analysis is applied when testing scenarios that involve limits or boundaries. In the unit testing section, it is used to ensure that the system responds correctly to the lower and upper boundaries of input values. For example, when a user selects a learning module, the system should respond correctly for values ranging from 0 to 10. This technique helps uncover issues related to handling values at the boundaries.

MC-DC Coverage: While not explicitly mentioned in the testing plan, achieving MC-DC coverage is the underlying goal of unit testing. Thorough unit testing, as described in the plan, aims to cover different conditions and decisions within the code. It ensures that various logical paths are tested independently, providing comprehensive coverage of the code's functionality. This approach helps identify potential issues with the code's logic.

Summary
Our QA plan covers a broad spectrum of testing activities to ensure CodeVenture meets its functional and non-functional requirements. We aim to deliver a high-quality and reliable software solution. Unit tests are designed to cover positive and negative scenarios, while functional testing includes thorough end-to-end tests that mimic real user interactions. This approach ensures that CodeVenture is thoroughly verified and ready for production use.

Overall Software Quality
Requirements alignment

REQ-1:
Requirement ‘Starting menu allows users to login or register’ satisfied

menu.py:
login_menu() - line 25 (login) & line 38 (register)

REQ-2
Requirement ‘Display username prompt’ satisfied.

login_system.py:
user_login() - line 45

REQ-3
Requirement “Display error message when a non-existing username is input’ not satisfied

login_system.py
user_login() - line 47

Menu brings user back to main menu instead of prompting for user input again

REQ-4
Requirement ‘Password prompt and learner’s username prompt if creating a parent account’ satisfied.

login_system.py
user_register() - line 77 to line 83

REQ-5
Requirement ‘null input should return user to home menu’ satisfied.

login_system.py:
user_login() - line 47

REQ-6
Requirement ‘Authenticate username and password’ satisfied.

login_system.py:
authenticate_user_login() - line 121

REQ-7
Requirement ‘Successful authentication results in login and menu options displayed’ satisfied.

menu.py:
login_menu() - line 29 to line 37

REQ-8
Requirement ‘Menu options displayed’ satisfied.

login_system.py:
print_menu() - line 38

menu.py:
learner_menu() - line 53 to line 56
parent_menu() - line 77 to line 81
educator_menu() - line 107 to line 110

REQ-9
Requirement ‘Menu displays all possible options for the particular user’ satisfied.

menu.py:
learner_menu() - line 53 to line 56
parent_menu() - line 77 to line 81
educator_menu() - line 107 to line 110

REQ-10
Requirement ‘System prompts user for menu option’ satisfied.

menu.py:

learner_menu() - line 60
parent_menu() - line 85
educator_menu() - line 114

REQ-11

Requirement ‘Validate user input’ satisfied
menu.py:

learner_menu() - line 58 to 63
parent_menu() - line 83 to line 88
educator_menu() - line 112 to line 117

REQ-12
Requirement ‘Invalid input displays error message’ satisfied.

menu.py:

learner_menu() - line 63
parent_menu() - line 88
educator_menu() - line 117

REQ-13
Requirement ‘Offer diverse range of learning modules’ work in progress.

learning_modules.py:
Learning_Modules Class __init__() - line 6
When a Learning_Modules object is initialised, lessons which are already predefined will be automatically added to the Learning_Modules object.

REQ-14
Requirement ‘Learning modules should be accessible from menu and categorized according to topic and difficulty’ not completely satisfied.

menu.py:
learner_menu() - line 66

Learning modules are accessible from the menu but are not categorized.

REQ-15
Requirement ‘Learning module provides clear instruction to concepts it covers’ satisfied.

learning_modules.py:
Line 73 to line 275 (Creation of lessons that will be added to learning modules by default)

REQ-16
Requirement ‘System provides content step-by-step’ satisfied.

learning_modules.py:
study_level() - line 31 to line 41

REQ-17
Requirement ‘Interactive quizzes shall test user understanding and provide immediate feedback’ satisfied.

learning_modules.py:
study_level() - line 37

REQ-18
Requirement ‘System shall track user progress included completed sections and quiz scores’ satisfied.

All methods defined under progress_tracker.py except “__init__()” and “display_progress()”

REQ-19
Requirement ‘System should always update progress of the current module’ not completely satisfied.

progress_tracker.py:
update_quiz_score() - line 14
update_challenge_score() - line 19

Progress tracker only displays when a module is completed but not when it is in progress.

REQ-20
Requirement ‘System shall generate challenges and quizzes related to each completed learning module’ satisfied

REQ-21
Requirement ‘System shall allow users to select any challenge or quiz through the menu’ satisfied.
(Quiz/Challenge can be accessed after selecting a learning_module)

REQ-22
Requirement ‘Challenges shall include concise instructions and sample input/output for easy understanding’ not yet implemented.

REQ-23
Requirement ‘User shall input solutions directly’ satisfied.

challenge.py:
start() - line 11

REQ-24
Requirement ‘System shall evaluate the input’s correctness based on the predefined answer’ not completely satisfied.

(System only evaluate answer is correct when answer is exactly the same)

REQ-25
Requirement ‘Upon receiving incorrect answers, the system shall display a hint and let the user try again’ not yet implemented

Incorrect answers result in proceeding to the next step.

REQ-26
Requirement ‘Upon receiving correct answers, the system shall provide positive feedback and acknowledge the achievement’ satisfied.

challenge.py:
start() - line 12 to line 14

‘Correct!’ message acknowledges the user's achievement.

REQ-27
Requirement ‘Progress tracker shall update after completing challenges/quizzes’ satisfied.

quiz.py:
Line 31 ‘learner.progress_tracker.update_quiz_score(level.level, score, num_of_questions)’

challenge.py:
Line 19 ‘learner.progress_tracker.update_challenge_score(level.level, score, num_of_questions)’

REQ-28
Requirement ‘Interface to access progress tracker from menu’ satisfied.

Learner users are provided an interface where they can input the corresponding option to access their progress tracker.

menu.py:
learner_menu() - line 67 to line 68

REQ-29
Requirement ‘Progress tracker shall provide a complete overview of all available learning modules’ satisfied.

Users can choose the option to display their progress on learning modules which shows the progress they have on each learning module’s quiz and challenge.

progress_tracker.py:
display_progress() - line 25 to line 29

REQ-30
Requirement ‘Progress tracker should display completion status’ not completely satisfied.

progress_tracker.py:
__init__() - line 10 to line 12
update_quiz_score() - line 16 to line 17
update_challenge_score() - line 21 to line 22

(Progress Tracker completion status either shows “Not Completed!” or the “number of question correct over the amount of question in that particular quiz” if attempted/completed but not “In Progress”)

REQ-31
Requirement “Users shall have the option to click on a module, challenge, or quiz to view more detailed progress.” not yet implemented.

Users can only view the amount of questions they answered correctly on each quiz or challenge.

REQ-32
Requirement ‘Recommendation based on learner’s progress’ not yet implemented

REQ-33
Requirement ‘Provide the learner with a menu upon logging in, displaying access to learning modules’ satisfied.

In line 46 of menu.py ‘def learner_menu(self, account):’, the function learner menu immediately displays the options the learner has.

REQ-34
Requirement ‘Progress tracker shows advancements’ satisfied.

In line 24 of progress_tracker.py ‘def display_progress(self):’, the function allows the user to view their progress on the quizzes and challenges.

REQ-35
Requirement ‘Learner can start a new learning module, continue an in-progress one, or review previously completed modules’ not completely satisfied.

In line 54 of menu.py ‘print("\t1. Select a learning module")’, learners can start a new learning module.

Learning modules cannot be paused so there is no option to continue an in-progress one.

In line 55 of menu.py ’print("\t2. Check your progress")’, learners can review previously completed modules as the progress tracker will show their score.

In line 20 of challenge.py ‘print(f'You have completed the "{level.name}" challenge and your score is: {score}/{num_of_questions}')’, learners are displayed the level they have completed and how well they did.

REQ-36
Not aligned with the requirement ‘Display learner’s username upon login’.

In line 30 of menu.py ‘Utils.display_str("Successfully logged in as a Learner!")’ only a successful message is displayed.

REQ-37
Requirement ‘Progress tracker displayed below child’s username’ satisfied.

In line 91 of menu.py ‘Utils.display_str(f"{account.learner.username}'s progress:")’ and line 92 ‘account.learner.progress_tracker.display_progress()’, the child’s username is displayed followed by their progress

NFR-1
Requirement ‘System should load within 5 seconds’ satisfied.

NFR-2
Requirement ‘System shall respond to user actions within 1 second’ satisfied.

NFR-3
Requirement ‘Database read and write operations should be completed within 100 milliseconds’ not yet satisfied.

Databases not completely implemented yet.

NFR-4
Requirement ‘The system should only collect essential user data’ satisfied.

The system only collects the bare minimum of user data to properly function such as username and password. Personal information is never collected.

NFR-5
Requirement ‘If content is posted, content moderation procedures shall be employed before releasing the post publicly’ not yet implemented.

NFR-6
Requirement ‘User chat shall filter certain words or phrases’ not yet implemented

NFR-7
Requirement ‘Parents shall be given parental controls to monitor or restrict their children’s usage of the system’ not completely satisfied

Parents being able to block and unblock their child’s access is the only parental control implemented.

NFR-8
Requirement ‘System should employ password complexity requirements’ not yet implemented

NFR-9
Requirement ‘System should allow for two-factor authentication to be enabled’ not yet implemented.

NFR-10
Requirement ‘If a user performs an action, the system shall log it accordingly with a timestamp’ not yet implemented.

NFR-11
Requirement ‘If a data breach occurs, the system shall contain the issue and warn its users’ not yet implemented.

NFR-12
Requirement ‘The system should be supported on major modern devices and browsers'

TBD

NFR-13
Requirement ‘The system should be easy to navigate even for beginners’ satisfied.

The system provides clear instructions for easy understanding and navigation.

NFR-14
Requirement ‘If an error occurs, the system shall display a clear error message’ satisfied.

NFR-15
Requirement ‘The system should have a failure rate of less than 1%’

TBD

Code Quality
Readability:
Our code does not contain comments explaining what each function does so readability is not satisfactory

Maintainability:
Our code utilizes modular programming which allows for easy maintainability

Comprehensive test coverage
The unit testing will be done by implementing some PyTest functions of which it tests the response of the system towards an invalid input or action from the user side.

Usability Considerations
Since our code at this point runs entirely on the terminal, it is not exactly intuitive. We plan to implement a graphical user interface which will greatly improve the user experience.

Fault Tolerance
Our system accounts for potential errors that may occur when unwanted inputs are given.

Revision
- CodeVenture needs to have GUI instead of running completely on the terminal as it will greatly improve user experience. This will be implemented with tkinter.
- Quizzes and challenges should have the option to quit midway through rather than completely finishing first to satisfy REQ-19 and REQ-30